

Python Programming

Unit 02 – Lecture 03: Sets and Dictionaries

Tofik Ali

School of Computer Science, UPES Dehradun

February 20, 2026

Repository: <https://github.com/tali7c/Python-Programming>

Quick Links

[Core Concepts](#)[Demo](#)[Interactive](#)[Summary](#)

Agenda

- 1 Overview
- 2 Core Concepts
- 3 Demo
- 4 Interactive
- 5 Summary

Learning Outcomes

- Explain why sets guarantee uniqueness and why they do not support indexing

Learning Outcomes

- Explain why sets guarantee uniqueness and why they do not support indexing
- Apply set methods (add/update/remove/discard) and set operations (union, intersection, etc.)

Learning Outcomes

- Explain why sets guarantee uniqueness and why they do not support indexing
- Apply set methods (add/update/remove/discard) and set operations (union, intersection, etc.)
- Build dictionaries for real-world key–value modeling and safe lookup

Learning Outcomes

- Explain why sets guarantee uniqueness and why they do not support indexing
- Apply set methods (add/update/remove/discard) and set operations (union, intersection, etc.)
- Build dictionaries for real-world key–value modeling and safe lookup
- Work with nested dictionaries/lists and sort dictionary items by key/value

Learning Outcomes

- Explain why sets guarantee uniqueness and why they do not support indexing
- Apply set methods (add/update/remove/discard) and set operations (union, intersection, etc.)
- Build dictionaries for real-world key–value modeling and safe lookup
- Work with nested dictionaries/lists and sort dictionary items by key/value
- Use typecasting (list/tuple/set/dict) for data cleaning and restructuring

Sets: Definition and Properties

- Unordered collection of **unique** elements

Sets: Definition and Properties

- Unordered collection of **unique** elements
- Fast membership testing: `x in s`

Sets: Definition and Properties

- Unordered collection of **unique** elements
- Fast membership testing: `x in s`
- Mutable (you can add/remove elements)

Sets: Definition and Properties

- Unordered collection of **unique** elements
- Fast membership testing: `x in s`
- Mutable (you can add/remove elements)
- Elements must be **hashable** (e.g., int/str/tuple, not list)

Creating Sets (Including Empty Sets)

- Use braces for non-empty sets: {1, 2, 3}

```
A = {1, 2, 2, 3}           # duplicates removed
B = set([3, 4, 4, 5])      # from list
empty = set()              # NOT {}
```

Creating Sets (Including Empty Sets)

- Use braces for non-empty sets: {1, 2, 3}
- Use `set(...)` to convert from any iterable

```
A = {1, 2, 2, 3}           # duplicates removed
B = set([3, 4, 4, 5])      # from list
empty = set()              # NOT {}
```

Creating Sets (Including Empty Sets)

- Use braces for non-empty sets: {1, 2, 3}
- Use `set(...)` to convert from any iterable
- `{}` is an empty dictionary, so use `set()` for an empty set

```
A = {1, 2, 2, 3}           # duplicates removed
B = set([3, 4, 4, 5])      # from list
empty = set()              # NOT {}
```

Set Methods (Add/Remove/Clear/Copy)

Method	What it does
<code>add(x)</code>	add one element
<code>update(iter)</code>	add many elements
<code>remove(x)</code>	remove; error if missing
<code>discard(x)</code>	remove; no error if missing
<code>pop()</code>	remove and return an arbitrary element
<code>clear()</code>	remove all elements
<code>copy()</code>	shallow copy

Set Operations: Operators vs Methods

■ Union: $A \mid B$ or `A.union(B)`

```
A = {1, 2, 3}
B = {2, 3, 4}
print(A & B)    # {2, 3}
```

Set Operations: Operators vs Methods

- Union: $A \mid B$ or `A.union(B)`
- Intersection: $A \& B$ or `A.intersection(B)`

```
A = {1, 2, 3}
B = {2, 3, 4}
print(A & B)    # {2, 3}
```

Set Operations: Operators vs Methods

- Union: $A \mid B$ or `A.union(B)`
- Intersection: $A \& B$ or `A.intersection(B)`
- Difference: $A - B$ or `A.difference(B)`

```
A = {1, 2, 3}
B = {2, 3, 4}
print(A & B)    # {2, 3}
```

Set Operations: Operators vs Methods

- Union: $A \mid B$ or `A.union(B)`
- Intersection: $A \& B$ or `A.intersection(B)`
- Difference: $A - B$ or `A.difference(B)`
- Symmetric difference: $A \wedge B$ or `A.symmetric_difference(B)`

```
A = {1, 2, 3}
```

```
B = {2, 3, 4}
```

```
print(A & B)    # {2, 3}
```

Set Comparisons (Subset/Superset/Disjoint)

- Subset: $A \leq B$ or `A.issubset(B)`

Set Comparisons (Subset/Superset/Disjoint)

- Subset: $A \leq B$ or `A.issubset(B)`
- Superset: $A \geq B$ or `A.issuperset(B)`

Set Comparisons (Subset/Superset/Disjoint)

- Subset: $A \leq B$ or `A.issubset(B)`
- Superset: $A \geq B$ or `A.issuperset(B)`
- Disjoint: `A.isdisjoint(B)` (no common elements)

Dictionaries: Key–Value Model

- Map keys to values (fast lookup by key)

```
student = {"name": "Asha", "marks": 88}  
print(student["name"])
```

Dictionaries: Key–Value Model

- Map keys to values (fast lookup by key)
- Keys must be unique and hashable (immutable)

```
student = {"name": "Asha", "marks": 88}  
print(student["name"])
```

Dictionaries: Key–Value Model

- Map keys to values (fast lookup by key)
- Keys must be unique and hashable (immutable)
- Values can be any type (including lists/dicts)

```
student = {"name": "Asha", "marks": 88}  
print(student["name"])
```

Dictionaries: Key–Value Model

- Map keys to values (fast lookup by key)
- Keys must be unique and hashable (immutable)
- Values can be any type (including lists/dicts)
- In modern Python, dictionaries preserve insertion order (but think “mapping”)

```
student = {"name": "Asha", "marks": 88}  
print(student["name"])
```

Creating Dictionaries (Common Ways)

- Curly braces: `{"a": 1, "b": 2}`

Creating Dictionaries (Common Ways)

- Curly braces: `{"a": 1, "b": 2}`
- `dict()` constructor: `dict(a=1, b=2)`

Creating Dictionaries (Common Ways)

- Curly braces: `{"a": 1, "b": 2}`
- `dict()` constructor: `dict(a=1, b=2)`
- From a list of pairs: `dict([("a", 1), ("b", 2)])`

Essential Dictionary Methods

- Views: `d.keys()`, `d.values()`, `d.items()`

```
freq = {}  
word = "python"  
freq[word] = freq.get(word, 0) + 1
```

Essential Dictionary Methods

- Views: `d.keys()`, `d.values()`, `d.items()`
- Safe access: `d.get(key, default)`

```
freq = {}  
word = "python"  
freq[word] = freq.get(word, 0) + 1
```

Essential Dictionary Methods

- Views: `d.keys()`, `d.values()`, `d.items()`
- Safe access: `d.get(key, default)`
- Update/merge: `d.update(other)`

```
freq = {}  
word = "python"  
freq[word] = freq.get(word, 0) + 1
```

Essential Dictionary Methods

- Views: `d.keys()`, `d.values()`, `d.items()`
- Safe access: `d.get(key, default)`
- Update/merge: `d.update(other)`
- Removal: `d.pop(key)`, `d.popitem()`

```
freq = {}  
word = "python"  
freq[word] = freq.get(word, 0) + 1
```

Essential Dictionary Methods

- Views: `d.keys()`, `d.values()`, `d.items()`
- Safe access: `d.get(key, default)`
- Update/merge: `d.update(other)`
- Removal: `d.pop(key)`, `d.popitem()`
- Cleanup: `d.clear()`

```
freq = {}  
word = "python"  
freq[word] = freq.get(word, 0) + 1
```

Nested Dictionaries and Lists

- Real data is often nested (JSON-like)

```
response = {"user": {"profile": {"email": "k@gmail.com"}  
email = response["user"]["profile"]["email"]
```

Nested Dictionaries and Lists

- Real data is often nested (JSON-like)
- Access step-by-step using keys and indices

```
response = {"user": {"profile": {"email": "k@gmail.com"}  
email = response["user"]["profile"]["email"]
```

Sorting Dictionary Data

- Sort by key: `sorted(d.items())`

```
counts = {"a": 3, "b": 1, "c": 2}
top = sorted(counts.items(), key=lambda kv: kv[1], re
```

Sorting Dictionary Data

- Sort by key: `sorted(d.items())`
- Sort by value: `sorted(d.items(), key=lambda kv: kv[1])`

```
counts = {"a": 3, "b": 1, "c": 2}
top = sorted(counts.items(), key=lambda kv: kv[1], re
```

Sorting Dictionary Data

- Sort by key: `sorted(d.items())`
- Sort by value: `sorted(d.items(), key=lambda kv: kv[1])`
- Ranking (top-k): sort then slice

```
counts = {"a": 3, "b": 1, "c": 2}
top = sorted(counts.items(), key=lambda kv: kv[1], re
```

Typecasting Between Collections (Data Cleaning)

- Remove duplicates: `list(set(data))` (order may change)

Typecasting Between Collections (Data Cleaning)

- Remove duplicates: `list(set(data))` (order may change)
- Modify an immutable tuple: `tuple -> list -> tuple`

Typecasting Between Collections (Data Cleaning)

- Remove duplicates: `list(set(data))` (order may change)
- Modify an immutable tuple: `tuple -> list -> tuple`
- Build dict from pairs: `dict([("a", 1), ("b", 2)])`

Where Sets and Dictionaries Shine

- Sets: unique email addresses, fast membership checks, comparing groups

Where Sets and Dictionaries Shine

- Sets: unique email addresses, fast membership checks, comparing groups
- Dictionaries: student records, inventory, shopping carts, frequency counts

Where Sets and Dictionaries Shine

- Sets: unique email addresses, fast membership checks, comparing groups
- Dictionaries: student records, inventory, shopping carts, frequency counts
- Both: building reliable, structured data models for programs

Demo: Word Frequency and Unique Words

- Split a sentence into words

Script: `demo/word_frequency.py`

Demo: Word Frequency and Unique Words

- Split a sentence into words
- Build a frequency dictionary (`get(..., 0)` pattern)

Script: `demo/word_frequency.py`

Demo: Word Frequency and Unique Words

- Split a sentence into words
- Build a frequency dictionary (`get(..., 0)` pattern)
- Build a set of unique words

Script: `demo/word_frequency.py`

Demo: Word Frequency and Unique Words

- Split a sentence into words
- Build a frequency dictionary (`get(..., 0)` pattern)
- Build a set of unique words
- Extension: print top 3 most frequent words

Script: `demo/word_frequency.py`

Checkpoint 1

Which code correctly creates an empty set?

1 `s = {}`

Checkpoint 1

Which code correctly creates an empty set?

1 `s = {}`

2 `s = set()`

Checkpoint 2

What is the difference between `s.remove(x)` and `s.discard(x)`?

Checkpoint 3

Given response = {"user": {"profile": {"email":
"k@gmail.com"}}}},
write the expression to access the email value.

Think-Pair-Share

Choose the right collection type (list/tuple/set/dict) for:

- unique roll numbers in a class

Think-Pair-Share

Choose the right collection type (list/tuple/set/dict) for:

- unique roll numbers in a class
- a fixed (x, y) coordinate

Think-Pair-Share

Choose the right collection type (list/tuple/set/dict) for:

- unique roll numbers in a class
- a fixed (x, y) coordinate
- a student's profile (name, roll, marks)

Summary

- Sets store unique, unordered values and support set algebra

Summary

- Sets store unique, unordered values and support set algebra
- Dictionaries map keys to values for fast lookup and counting

Summary

- Sets store unique, unordered values and support set algebra
- Dictionaries map keys to values for fast lookup and counting
- Nested structures are common in real data (JSON-like)

Summary

- Sets store unique, unordered values and support set algebra
- Dictionaries map keys to values for fast lookup and counting
- Nested structures are common in real data (JSON-like)
- Sorting and typecasting help with analytics and data cleaning

Exit Question

Write one line to sort dictionary items by value in descending order.